WEEK-8---SNS,SQS & S3-SNS-SQS-LAMBDA SERVICES

1) Simple Notification Sevice (SNS)

A) Create SNS Topic and Send Message to Email

We will:

1. Create an SNS Topic
2. Create an Email Subscription
3. Confirm the subscription
4. Publish a test message

Step 1: Create SNS Topic

1. Login to **AWS Console**
2. In search bar → Type **SNS**
3. Open **Amazon SNS (Simple Notification Service)**
4. In left panel → Click **Topics**
5. Click **Create topic**

Configure:

- **Type** → Select **Standard**
- **Name** → MyEmailTopic
- Leave other settings default (unless required)

6. Click **Create topic**

Step 2: Create Email Subscription

1. Open your created topic
2. Go to **Subscriptions** tab
3. Click **Create subscription**

Configure:

- **Protocol** → Select **Email**
- **Endpoint** → Enter your email address

4. Click **Create subscription**

Step 3: Confirm Subscription

- Go to your email inbox
- You will receive a mail from **Amazon SNS**
- Click **Confirm subscription**

Now your email is successfully subscribed 🎉

Step 4: Publish Test Message

1. Open your SNS topic
2. Click **Publish message**
3. Add:
    - **Subject** → Test Mail
    - **Message body** → Hello this is SNS test
4. Click **Publish message**

You will receive email notification.

B) Trigger Email When S3 Object is Uploaded (S3 → SNS → Email)

Now we extend this setup:

Flow:

S3 Bucket → Event Notification → SNS Topic → Email

Step 1: Create S3 Bucket

1. Search → **S3**
2. Click **Create bucket**

Configure:

- **Bucket name** → my-upload-bucket-2026
- Choose Region (Important: Same region as SNS topic)
- Keep default settings (unless specific requirement)

3. Click **Create bucket**

Step 2: Create SNS Topic (If not already created)

You can reuse the topic from Part A
OR create new topic like:

- Topic name → S3UploadNotification

Make sure email subscription is confirmed.

Step 3: Allow S3 to Publish to SNS (Access Policy)

1. Open SNS Topic
2. Go to **Access policy**
3. Click **Edit**

Add permission for S3 to publish.

If using Basic Editor:

Access Policy that needs to be configured on SNS:

a) replace SNS topic ARN

b) Replace s3 bucket ARN

c) Replace account

```
{
  "Version": "2012-10-17",
  "Id": "example-ID",
  "Statement": [
    {
      "Sid": "Example SNS topic policy",
      "Effect": "Allow",
      "Principal": {
        "Service": "s3.amazonaws.com"
      },
      "Action": [
        "SNS:Publish"
```

```
    ],
    "Resource": "SNS topic ARN",
    "Condition": {
      "ArnLike": {
        "aws:SourceArn": "s3 bucket ARN"
      },
      "StringEquals": {
        "aws:SourceAccount": "account id"
      }
    }
  }
 ]
}
```

(Modern AWS console often auto-generates this when configuring from S3)

Step 4: Configure S3 Event Notification

1. Open your S3 bucket
2. Go to **Properties**
3. Scroll to **Event notifications**
4. Click **Create event notification**

Configure:

- **Event name** → UploadNotification
- **Event types** → Select:
  - All object create events
- Destination:
  - Select **SNS topic**
  - Choose your topic (S3UploadNotification)

5. Click **Save changes**

Step 5: Test It

1. Upload any file into the S3 bucket
2. After upload completes:
    - S3 triggers event
    - SNS receives publish request
    - SNS sends email

You will receive mail about object upload.

Simple Queue Service (SQS)

2) Create SQS Queue and Send Message

We will:

1. Create an SQS Queue
2. Send a message to the Queue
3. Receive the message from the Queue

Step 1: Create SQS Queue

1. Login to **AWS Console**
2. In the **search bar → Type SQS**
3. Open **Amazon Simple Queue Service**
4. In the dashboard → Click **Create queue**

Configure:

• **Type → Select Standard**
• **Name → MyQueue**
• Leave other settings as **default**

5. Click **Create queue**

Your **SQS queue is now created successfully.**

Step 2: Send Message to Queue

1. Open the **MyQueue** queue
2. Click **Send and receive messages**
3. In the **Message body**, type:

Hello this is SQS test message

4. Click **Send message**

Your message is now stored in the queue.

Step 3: Receive Message from Queue

1. In the same page (**Send and receive messages**)
2. Click **Poll for messages**

You will see the message displayed.

Example output:

Hello this is SQS test message

3. After processing, click **Delete message** (optional)

Result:

- SQS queue created
- Message sent successfully
- Message received from queue

3) S3 Bucket Event → SNS → SQS → Lambda Processing

S3 bucket event (like file upload) triggers SNS, which sends the notification to SQS. Then **Lambda acts as the consumer and processes the message**.

Services used:

• Amazon S3
• Amazon Simple Notification Service
• Amazon Simple Queue Service
• AWS Lambda

S3 → SNS → SQS → Lambda Architecture

S3 Bucket (File Upload)

↓

SNS Topic

↓

SQS Queue

↓

Lambda Function (Consumer)

When a file is uploaded, S3 sends an event notification to SNS, which forwards the message to SQS. The Lambda function then reads and processes the message from SQS.

Step 1: Create S3 Bucket

1. Login to AWS Console
2. Search **S3**
3. Open **Amazon S3**

4. Click **Create bucket**

Configure:

• Bucket name → **mys3eventbucket123** (must be unique)
• Region → choose your region

Leave other settings default.

5. Click **Create bucket**

Step 2: Create SNS Topic

1. Search **SNS** in AWS Console
2. Open **Amazon SNS**
3. Click **Topics → Create topic**

Configure:

• Type → **Standard**
• Name → **MyS3SNSTopic**

4. Click **Create topic**

Copy the **SNS Topic ARN**.

Step 3: Create SQS Queue

1. Search **SQS** in AWS Console
2. Open **Amazon SQS**
3. Click **Create queue**

Configure:

• Type → **Standard**
• Name → **MyS3Queue**

4. Click **Create queue**

Copy the **Queue ARN**.

Step 4: Subscribe SQS to SNS

1. Open **MyS3SNSTopic**

2. Click **Create subscription**

Configure:

• Protocol → **Amazon SQS**
• Endpoint → Select **MyS3Queue**

3. Click **Create subscription**

Step 5: Add Access Policy to SQS

Open **MyS3Queue → Access Policy → Edit**

Replace:

• SQS ARN
• SNS ARN

```
{
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "sns.amazonaws.com"
      },
      "Action": "sqs:SendMessage",
      "Resource": "SQS ARN",
      "Condition": {
        "ArnEquals": {
          "aws:SourceArn": "SNS ARN"
        }
      }
    }
```

```
  ]
}
```

(Modern AWS console often auto-generates this when configuring from S3)

Click **Save**.

Step 6: Add Access Policy to SNS

Open **SNS Topic → Edit access policy**

Replace:

• SNS topic ARN
• S3 bucket ARN
• Account ID

```
{
  "Version": "2012-10-17",
  "Id": "example-ID",
  "Statement": [
    {
      "Sid": "Example SNS topic policy",
      "Effect": "Allow",
      "Principal": {
        "Service": "s3.amazonaws.com"
      },
      "Action": [
        "SNS:Publish"
      ],
```

```
    "Resource": "SNS topic ARN",
    "Condition": {
      "ArnLike": {
        "aws:SourceArn": "S3 bucket ARN"
      },
      "StringEquals": {
        "aws:SourceAccount": "Account ID"
      }
    }
  }
 ]
}
```

Save the policy.

Step 7: Configure S3 Event Notification

1. Open **S3 Bucket (mys3eventbucket123)**
2. Go to **Properties**
3. Scroll to **Event notifications**

Click **Create event notification**

Configure:

• Event name → **S3UploadEvent**
• Event type → **All object create events**

Destination:

• **SNS Topic → MyS3SNSTopic**

Click **Save changes**.

Step 8: Test the Architecture

1. Open the **S3 bucket**
2. Click **Upload file**
3. Upload any file (example: **test.txt**)

Now:

File Upload → S3

↓

Notification → SNS

↓

Message → SQS

Step 9: Verify Message in SQS

1. Open **MyS3Queue**
2. Click **Send and receive messages**
3. Click **Poll for messages**

You will see the **S3 event notification message**.

Step 10: Configure Lambda as Consumer

Now configure Lambda to **automatically process messages from SQS**.

Create Lambda Function

1. Search **Lambda** in AWS Console
2. Open **AWS Lambda**
3. Click **Create Function**

Configure:

• Author from scratch
• Function name → **SQSConsumerFunction**
• Runtime → **Python 3.x**

Click **Create function**

Add SQS Trigger

1. Open **SQSConsumerFunction**
2. Click **Add Trigger**
3. Select **SQS**
4. Choose **MyS3Queue**

Click **Add**

Now Lambda will automatically read messages from SQS.

Lambda Code

Replace code with:

```
def lambda_handler(event, context):


    for record in event['Records']:

        print("Message received from SQS:")

        print(record['body'])


    return {

        'statusCode': 200,

        'body': 'Message processed successfully'

    }
```

Click **Deploy**.

Final Architecture

User uploads file

↓

Amazon S3

↓

SNS Topic

↓

SQS Queue

↓

Lambda Function (Consumer)

↓

Processing / Logs / Database

Result

When a file is uploaded to S3:

1. S3 sends a notification to SNS
2. SNS forwards the message to SQS
3. SQS stores the message
4. Lambda automatically reads and processes the message

This creates a **serverless event-driven architecture commonly used in real cloud applications**.

Scenario Based Questions (SNS, SQS, S3-SNS-SQS-Lambda)

1. A company wants to send notifications to multiple users whenever an important system event occurs. The same message must be delivered to email, SMS, and application subscribers simultaneously.
   Which AWS service can be used to implement this publish-subscribe messaging system?
2. A developer created an SNS topic and added an email subscription, but the subscriber is not receiving any notifications.
   What could be the possible reason for this issue?
3. An organization wants to automatically notify administrators via email whenever a new file is uploaded to an S3 bucket.
   How can this be implemented using AWS services?
4. A developer configured an S3 bucket to trigger an SNS notification on file upload, but the message is not reaching the SNS topic.
   What configuration or permission issue might cause this problem?
5. A company wants to store incoming messages temporarily so that applications can process them later without losing data even if the system is busy.
   Which AWS messaging service should be used for this requirement?

6. A developer sends messages to an SQS queue, but when checking the queue no messages appear.
   What possible reasons could cause this issue?
7. A company wants to process thousands of messages coming from different applications without overloading the processing system.
   How does Amazon SQS help in handling this situation?
8. A developer wants to ensure that messages are processed automatically from an SQS queue without manually polling the queue.
   Which AWS service can be configured to automatically consume messages from SQS?
9. An application uploads files to S3, and the company wants the upload event to trigger a workflow that sends notifications and processes the message later.
   How can this be designed using S3, SNS, SQS, and Lambda?
10. A developer configured an SNS topic to send messages to an SQS queue, but the queue is not receiving any messages.
    What policy configuration might be missing?
11. A company wants to decouple its application components so that failures in one component do not affect others.
    Which AWS service combination can be used to implement loosely coupled communication?
12. A Lambda function is configured to process messages from an SQS queue, but the function is not being triggered.
    What possible configuration issue might cause this problem?
13. An organization wants to ensure that uploaded files in S3 trigger notifications, store messages reliably, and process them asynchronously.
    Which serverless architecture using AWS services can achieve this?
14. A developer wants to monitor the processing of messages handled by a Lambda function triggered by SQS.
    Which AWS service can be used to view logs and troubleshoot the function execution?
15. A company wants to build a fully serverless event-driven system where file uploads automatically trigger notifications and background processing without managing servers.
    Which AWS services can be combined to build this architecture?

16. • A company wants to send a notification to multiple applications whenever a new order is placed in their system. Each application should receive the same message and process it independently.
    Which AWS service can be used to implement this publish-subscribe communication model?
17. • A developer configured an SNS topic and connected it to an SQS queue, but the messages published to SNS are not appearing in the queue.
    What configuration or permission might be missing in this setup?
18. • An application generates a large number of events, and the development team wants to ensure that these events are stored safely until they are processed by another service.
    Which AWS service can be used to reliably store and deliver these messages?
19. A company uploads files to an S3 bucket and wants to process these files using Lambda, but only after the messages are safely stored to avoid data loss during high

traffic.
Which AWS architecture using SNS and SQS can help achieve this requirement?

20. A developer notices that messages remain in an SQS queue even after being processed by a Lambda function.
What configuration or step might be missing to ensure messages are removed after processing?